

Polars + DuckDB: The New Power Combo for In-Process Analytics

Polars and DuckDB form an excellent in-process analytics stack for 2026. They occupy the important middle ground between traditional DataFrame libraries and fully distributed systems, offering high performance without operational overhead.



Over the last decade, distributed data processing frameworks, such as Apache Spark, have been the default solution for analytics workloads that exceed the limits of traditional DataFrame tools. Many teams are now realising that a large share of their analytics jobs operate on tens of gigabytes, not terabytes, and run on machines that are far more capable than those of a decade ago.

In-process analytics is gaining traction because modern laptops and cloud VMs routinely offer 32-128 GB of RAM, fast NVMe storage, and many-core CPUs. For mid-scale ETL, feature engineering, and analytical reporting, the bottleneck is often not raw compute power but system complexity.

This has driven a shift away from heavyweight distributed systems towards lightweight, local analytics engines that start instantly, are easier to debug, and integrate naturally into application code. Polars and DuckDB exemplify this shift.

Overview of Polars and DuckDB

Polars is a high-performance DataFrame library written in Rust with bindings for Python and other languages. While often compared to Pandas, Polars is fundamentally different in its execution model. It is built on the Apache Arrow columnar format, uses multi-threaded execution by default, and emphasises lazy evaluation. Instead of executing operations eagerly, Polars can construct a full query plan and optimise it before touching data.

DuckDB is an embedded analytical database optimised for OLAP-style workloads. Often described as ‘SQLite for

analytics’, DuckDB runs entirely in-process and requires no separate server. It provides a rich SQL dialect, vectorised execution, and efficient scanning of columnar data formats such as Parquet.

These two tools complement each other well. Polars excels at transformation-heavy, programmatic logic such as feature engineering and schema enforcement. DuckDB shines when expressing relational operations, such as joins, aggregations, and analytical queries, in SQL. Because both systems are designed around columnar data and integrate with Arrow-based memory, data can move between them efficiently without unnecessary serialisation.

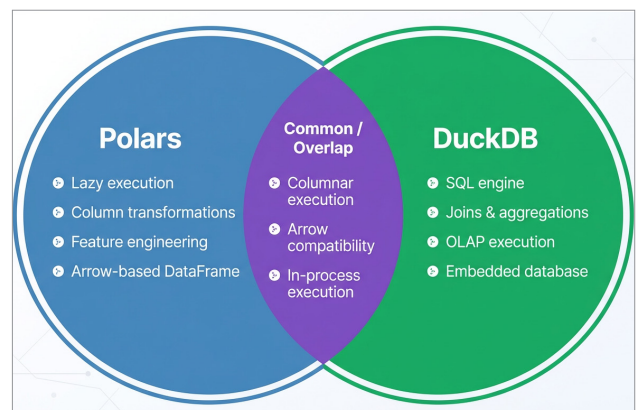


Figure 1: Polars and DuckDB comparison